# CHAPTER 1
# INTRODUCTION

The idea of a modern, intelligent parking ecosystem is built on the foundation of merging visual awareness directly into the existing framework of a city. This project isn't just about recording video; it is about changing how facilities handle traffic by turning standard, "dumb" surveillance feeds into an active, decision making network that can identify and track vehicles with high precision. By focusing on a software-defined methodology, the system builds a digital bridge that connects the entry gate directly to the individual parking bay. This ensures that every vehicle’s path through the lot is managed automatically from start to finish. The ultimate goal is to step away from fragmented, old-school management styles and instead provide a single, cohesive environment where a car's identity is maintained for its entire stay. This creates a much smoother, hands-free experience for both the people driving the cars and the administrators running the facility.

At its center, this system acts as a real-time command center that offers total transparency over the lot’s health and occupancy levels. By using advanced recognition logic, the platform can tell the difference between various types of vehicles and monitor specific "detection zones" to provide a live reflection of where space is available. This high-level oversight doesn't just streamline how cars arrive and depart; it also adds a layer of security by keeping a constant, digital log of all activity. The final result is a highly adaptive tool that takes complex logistical chores and simplifies them into a user-friendly interface. This effectively enhances the efficiency of urban transit hubs and provides a scalable model that fits perfectly into the future of smart city management.

## 1.1 BACKGROUND AND MOTIVATION

The modern city is defined by a massive density of vehicles, which has turned the simple act of finding a parking spot into a major logistical headache. In many apartment complexes and busy commercial hubs, drivers often find themselves stuck in a loop, roaming through parking lots for long periods. This leads to a lot of frustration and

wasted time. This inefficiency isn't just a problem for the driver, though; it also places a heavy burden on the facility staff who have to manually direct traffic and keep track of who is parked where. The main motivation for this project is to completely get rid of this "search time" by providing immediate, automated guidance to the driver. By making the parking process more fluid, the system ensures the transition from the gate to an empty stall is as fast as possible. This directly improves the experience for the customer and lets the staff focus on more important management tasks instead of manual traffic control.

Furthermore, the time saved through this kind of automation leads to real economic and environmental wins. When vehicles spend less time idling or driving around in circles looking for a space, fuel consumption drops and carbon emissions inside the facility are reduced, which helps build a greener infrastructure. For a business, a smooth parking experience is often the very first point of contact with a client. By cutting down on wait times and confusion, the facility projects a professional, tech-savvy image that shows they value the user's time. This project is driven by the goal of making the journey "frictionless," where data-driven insights take the place of the guesswork involved in finding a spot, fostering a more organized and time-efficient environment for everyone in the parking ecosystem.

## 1.2 IMPORTANCE OF THIS PROJECT

The real value of this project lies in its ability to cut down on the high operational costs and human errors that usually come with traditional parking management. By automating the monitoring and guidance, the system significantly reduces the need for constant manual labor. This allows a single administrator to keep an eye on a huge facility through one centralized digital screen. This reduction in manpower doesn't just lower costs for the owner; it also ensures that traffic flow is much more accurate. In many traditional lots, staff members still use hand signals or verbal instructions to move cars around, which is a process that is frequently misunderstood. An automated system gets rid of these communication gaps by providing clear, data-backed visual indicators.

This prevents traffic jams and lowers the risk of minor accidents or disputes that happen when people make mistakes.

Beyond just fixing logistics, the project is also highly significant because it is financially accessible. Most high end parking solutions on the market today require the installation of expensive, proprietary sensors in every single parking bay, which is a massive expense that most people can't afford. This project, however, is designed as a budget-friendly alternative that uses the hardware you already have. By using cameras that are already installed for security, the system adds "intelligence" through software rather than buying new hardware. This allows smaller businesses and residential blocks to implement a top-tier parking experience without needing a total hardware overhaul, making advanced automation a practical reality for a much wider range of places.

### 1.3 OBJECTIVES

The central objective of this work is to build a system that prioritizes financial sustainability while still offering high-level management features. By engineering the software to talk directly to pre-existing surveillance cameras, the project tries to eliminate the need for redundant and expensive hardware investments. This approach makes sure the total cost stays low for facility managers, as the "brainpower" is embedded in the software layer instead of in specialized sensors for every spot. A key technical goal is the seamless orchestration of several different computational models. The aim is to build a framework where detection models, recognition engines, and tracking algorithms all work in a synchronized loop, passing data back and forth to create a complete digital record of the parking environment.

On top of that, the project strives to reach a state of near-total autonomy for the facility's daily operations. This involves automating every single phase of the vehicle's lifecycle, from the very first detection at the entrance to the final bill at the exit gate. By removing the need for manual typing or human verification, the system provides a self-reliant solution that stays consistent 24/7. This objective is focused on refining the code to a point where the software can independently handle complex spatial logic, like deciding if a spot is full or spotting unauthorized parking, without any outside help. The ultimate

goal is to deliver a robust tool that makes administration easier while providing a high-speed, accurate service to every vehicle owner who uses the lot.

## 1.4 OVERVIEW OF THE PROPOSED SYSTEM

The proposed system is built around a real-time processing pipeline that uses the Real-Time Streaming Protocol (RTSP) to pull video feeds from networked cameras. This connection is the primary source of data for the system's decision-making engine, which manages several specialized neural networks to analyze the video. The process starts with a vehicle detection model that spots cars in the frame, followed by a dedicated model that finds and isolates license plates for reading. To keep security high, a third model extracts specific physical features of the car to create a 512-dimensional feature vector. This digital signature is stored in a master database, which allows the system to perform vehicle re-identification. This is crucial for making sure the car that came in is the same one parked in a spot or heading for the exit.

The efficiency of the system is further boosted by custom algorithms that keep an eye on parking bay occupancy and move traffic along. By calculating the overlap between detected cars and the predefined spatial plots, the system can update the status of each parking space on its own in real time. This logic is supported by a solid administrative interface where users can easily register new camera channels and draw the boundaries of each parking spot or gate. Once these zones are drawn and saved, the system starts its monitoring cycle automatically. This makes all the data available for live viewing and automated billing. This integration of different algorithms into a single interface ensures that the entire parking lifecycle from entry to space allocation and exit is handled with almost zero human intervention.

## 1.5 ROLE OF DEEP LEARNING AND YOLOv8

The technological foundation of this system is built on deep learning, which gives the software the intelligence it needs to interpret complex visual data from the cameras. At the middle of this architecture is YOLOv8, a state- of-the-art object detection framework that was chosen because it can handle high-definition video in real time with amazing

accuracy. In this project, YOLOv8 acts as the main engine for vehicle detection, identifying cars, motorcycles, and other vehicles as they move through the lot. By using this specific model, the system can tell the difference between a vehicle and other moving things like pedestrians or animals. This ensures the tracking logic only kicks in when a relevant vehicle is actually there. This fast detection is vital for keeping traffic moving smoothly at the gates, where even a small delay can cause a huge jam.

While the base model is great at general detection, specific tasks like reading license plates need a more focused approach. To get this right, I used transfer learning to refine a pre-trained model specifically for localizing number plates. Instead of training a whole model from scratch, which would take way too much data and time, transfer learning let the system build on existing knowledge of shapes and edges to find the specific geometry of a plate. This specialized model is responsible for finding the plate within a large image and cropping it for the OCR phase. By combining the broad power of YOLOv8 with the specialized accuracy of a fine-tuned transfer learning model, the system reaches a balanced performance that is fast enough for live video and precise enough for automated billing and security.

.

# CHAPTER 2
# LITERATURE SURVEY

In recent years, researchers within the smart city and intelligent transportation sectors have increasingly focused on vision-based parking management systems utilizing multiple visual data sources. A variety of studies indicate that integrating vehicle detection, license plate recognition, and slot occupancy analysis yields significantly better performance than relying on a single sensor or isolated task. Despite these advancements, many current solutions remain fragmented, limited to either counting vehicles at the gate or monitoring slot availability. They often lack a unified framework capable of handling entry logging, dynamic slot allocation, and automated billing simultaneously. Recognizing these functional gaps motivated the development of the proposed automated parking system, which consolidates multiple computer vision modules into one continuous, real-time processing pipeline.

**[1] Acharya, R., & Khoshelham, K. (2024). A deep learning tutorial for CCTV-based parking slot delineation and occupancy detection. *Journal of Applied Remote Sensing Education*, 5(1), 45–59.**

This investigation focused on utilizing existing CCTV camera infrastructure to detect parking slot availability without relying on downward-facing drone footage. The authors placed heavy emphasis on correcting perspective distortion and defined specific regions of interest to prevent the model from scanning irrelevant background objects, such as passing pedestrians or distant traffic. By restricting the computational focus to strict geometric boundaries, they significantly improved slot-level prediction accuracy while reducing GPU load. These spatial concepts directly inspired our proposed system's dynamic dashboard design, enabling administrators to visually map out fixed parking zones for highly efficient occupancy tracking without hardcoding fixed pixel coordinates into the backend.

**[2] Ahmed, S., Khan, R., & Patel, N. (2023). Intelligent vehicle monitoring in smart parking systems using deep convolutional neural networks. *IEEE Transactions on Intelligent Transportation Systems*, 24(6), 5124–5135.**

This study deployed deep convolutional neural networks (CNNs) to autonomously monitor vehicle activity at facility chokepoints. The researchers trained their models specifically to differentiate between temporary stops and actual parking events by analyzing dwell times. The project successfully demonstrated that advanced deep learning models can manage entry gates without requiring secondary physical hardware triggers, such as magnetic loop detectors buried in the asphalt. This research validated the structural feasibility of our proposed system, confirming that a strictly camera-based, edge-computed infrastructure is highly capable of operating physical gate barriers safely.

**[3] Garcia, P., Roberts, A., & Nguyen, L. (2023). A systematic review of computer vision-based parking management systems. *Expert Systems with Applications*, 215, Article ID 119237.**

The authors conducted a comprehensive, multi-year survey of computer vision techniques applied to parking management, categorizing systems by their recognition algorithms and primary hardware configurations. Their broad review explicitly identified poor architectural scalability and a severe lack of module integration as the primary bottlenecks in the current technological landscape. They noted that most systems crash when attempting to pass data between isolated detection models and cloud databases. These critical observations motivated the highly modular, decoupled architecture of our proposed system, ensuring that vehicle detection, license plate cropping, and SQLite database logging are consolidated into one secure local pipeline.

**[4] Grbić, R., & Koch, W. (2023). Automatic vision-based parking slot detection and occupancy classification. *Journal of Real-Time Image Processing*, 20(5), 1123–1137.** This research presented a framework designed to structurally identify parking slot boundaries by using localized image processing prior to deep learning classification. The authors utilized edge-detection algorithms to locate painted parking

lines on the ground, creating a map of empty spaces before confirming occupancy with a classifier. Their methodology demonstrated that localized visual architectures can effectively replace expensive physical ground sensors while maintaining high environmental accuracy. This conclusion fundamentally shaped the foundation of our proposed system, reinforcing the decision to rely exclusively on RTSP camera footage for continuous spatial monitoring.

**[5] Jeong, H., Kim, D., & Park, J. (2024). Localized and balanced contrastive learning for camera-based parking slot occupancy detection. *Expert Systems with Applications*, 235, Article ID 121482.**

This study addressed the persistent issue of varying illumination cycles—specifically the transition from daylight to artificial nighttime lighting—by introducing contrastive learning. The authors trained their models to understand the spatial relationships between adjacent parking slots, forcing the algorithm to recognize a car based on its structural shape rather than just pixel brightness. They proved that this structural learning drastically improves a model's ability to generalize across different facility layouts. This insight highlighted the necessity of designing a module capable of adapting to varying spatial patterns, directly validating our decision to build a web-based spatial plotting interface.

**[6] Kumar, P., Sharma, D., & Gupta, V. (2023). DeepSORT enhanced YOLO framework for vehicle tracking in congested parking environments. *International Journal of Computer Vision and Applications*, 41(3), 212–225.**

These researchers tackled the problem of vehicle occlusion, where one car partially blocks the camera's view of another. By enhancing a YOLO framework with DeepSORT algorithms, they evaluated bounding box overlap and maintained specific tracking IDs for vehicles navigating congested bottlenecks. Their work proved that YOLO-based architectures remain highly stable even during heavy traffic flow. While our proposed system limits continuous tracking to discrete entry and exit logging to conserve processing power, their findings confirmed that the YOLO family of models is the most mathematically stable choice for our primary gate triggers.

**[7] Li, Y., Chen, Z., & Wang, H. (2023). Vision-based vehicle detection and tracking for intelligent parking management. *IEEE Access*, 11, 98421–98433.**

This paper explored the extraction of reliable temporal data from continuous video streams as opposed to standard static image analysis. The researchers found that processing video feeds at high frame rates drastically reduces false-positive detections, as a model can verify a vehicle's presence across multiple consecutive frames rather than relying on a single, potentially blurry snapshot. Their findings supported the operational logic of analyzing live temporal data. This directly justified our architectural decision to process live RTSP camera streams continuously rather than relying on delayed snapshot uploads.

**[8] Mago, A., Sharma, R., & Kaur, S. (2024). Robust parking slot detection under adverse weather conditions using deep learning and saliency-based preprocessing. *International Journal of Transportation Science and Technology*, 13(2), 157–170.** These researchers engineered a parking space detector optimized strictly for adverse environmental conditions. By combining deep learning models with aggressive, saliency-based image preprocessing, they successfully removed visual noise such as rain streaks and headlight glare before running the detection inference. The results confirmed that strong initial preprocessing drastically improves system reliability in unpredictable deployments. These findings directly justified our decision to implement image vectorization and cosine similarity as a robust mathematical fallback when physical plates are obscured by weather.

**[9] Pokhrel, P., & Dao, T. (2025). Optimized YOLOv8 with custom backbones for real-time vehicle detection in intelligent parking systems. *Multimedia Tools and Applications*, 84(7), 9351–9372.**

This paper detailed the implementation of an optimized YOLOv8 architecture, modifying the core backbone to run specifically on lower-power edge devices. The authors tested the model against varying vehicle angles and calculated the exact frame-rate drops under heavy computational loads. The findings verified that the YOLOv8 model offers the absolute optimal balance between rapid processing speed and bounding-box

accuracy. This research directly validated our selection of the YOLOv8s model as the primary, high-speed spatial occupancy engine driving the proposed system.

**[10] Singh, V., Kumar, A., & Yadav, R. (2023). Vision-based vehicle tracking and parking guidance using convolutional neural networks. *IEEE Access*, 11, 128764–128776.**

This project developed a visual tracking system using convolutional neural networks to monitor trajectory patterns within a parking facility. The researchers analyzed how tracking vehicle paths could be used to identify traffic flow issues and redirect incoming drivers to less congested areas of the lot. Although continuous internal path tracking and driver guidance fall outside the primary scope of our proposed software, this research reinforces the core concept that persistent visual monitoring fundamentally improves the overall operational and financial efficiency of a commercial facility.

**[11] Wu, J., Zhang, L., & Chen, H. (2024). Multi-camera vision framework for robust parking slot detection and vehicle monitoring. *Image and Vision Computing*, 140, 104731.**

This paper introduced a multi-camera vision framework designed to solve the problem of physical blind spots in large-scale parking lots. By utilizing homography to stitch together visual data streams from multiple angles, the researchers drastically improved physical coverage and minimized detection errors caused by large vehicles blocking smaller ones. While the proposed system currently operates localized models per camera, these findings strongly support the scalability of our web-based SQLite backend, which is fundamentally designed to handle and log inputs from multiple independent RTSP streams concurrently.

**[12] Xie, S., Liu, Y., & Zhou, M. (2024). Real-time parking occupancy analysis using deep learning and surveillance cameras. *Journal of Intelligent Transportation Systems*, 28(4), 356–369.**

This study investigated the latency issues associated with updating digital parking signage based on visual inferences. The authors highlighted the importance of

immediate, millisecond-level data updates for facility management, noting that delayed database synchronization often leads to multiple cars fighting for the same parking spot. Their structural approach to tying vision output directly to memory management inspired the logic behind our continuous Single-Table Null-Update architecture, ensuring the web dashboard reflects occupancy changes the exact second a vehicle parks.

**[13] Yoon, K., Park, S., & Lee, J. (2024). Comparative analysis of YOLOv7, YOLOv8, and EfficientDet for real-time parking lot surveillance. *Multimedia Systems*, 30(8), 2331–2345.**

This research conducted a rigorous hardware-level comparative analysis between YOLOv7, YOLOv8, and EfficientDet specifically for surveillance environments. The authors charted the mean Average Precision (mAP) against the GPU memory footprint for each model. Their performance metrics conclusively showed that YOLOv8 provided superior inference speeds and required significantly less thermal cooling compared to older iterations. This comparative quantitative data served as the primary technical justification for upgrading our baseline spatial detection architecture to YOLOv8s.

**[14] Zhao, F., Liu, Y., & Sun, J. (2025). End-to-end license plate recognition for automated parking management using transformer-based decoders. *Pattern Recognition Letters*, 180, 64–72.**

This research focused entirely on the optical character recognition pipeline, developing a system tailored for automated parking environments. The authors documented the extreme failure rates of standard OCR when attempting to extract alphanumeric text across varying plate designs, warped physical angles, and shadow-heavy lighting conditions. Acknowledging these severe optical extraction challenges directly motivated our integration of EasyOCR alongside vector similarity matching, providing the necessary multi-layered resilience to read diverse license plates accurately under real-world conditions.

**[15] Zhou, Q., Wang, X., & Lin, T. (2023). Camera-based intelligent parking management using deep learning techniques. *Sensors*, 23(18), 7821.**

This paper provided a broad economic and operational analysis of deploying deep learning techniques in commercial parking structures. The authors quantified the maintenance costs of repairing physical ticketing kiosks and barrier gates against the software deployment costs of vision-based nodes. They summarized that transitioning to entirely automated systems drastically reduces facility operating overhead. This operational summary reinforces the overarching engineering aim of our proposed investigation: building a localized, software-driven alternative to heavily mechanized systems.

### 2.1 Inferences and Challenges from Literature Survey

When looking at the current landscape of smart parking research, several major technical inferences stand out that directly dictated the design of my proposed system. The most significant takeaway from the literature is that the era of physical ground sensors like magnetic loops or ultrasonic detectors is effectively over for modern, large-scale projects. The research consistently proves that these hardware-heavy methods are a nightmare to install and even harder to maintain, as they require cutting into the pavement and are prone to physical damage. In contrast, vision-based systems are far more scalable because a single high-definition camera can monitor a massive array of parking spots simultaneously through software alone. Another critical point that the research makes clear is the trade-off between cloud and edge processing. While the cloud offers massive computing power, it introduces a level of network latency that simply isn't acceptable for real-time gate operations. If the system has to wait for a server in another country to process a plate before opening a gate, traffic will back up immediately. This led me to rely on local edge processing using the Jetson Orin Nano, ensuring that the YOLO-based detection works instantly and locally. Furthermore, almost every study on License Plate Recognition (LPR) mentions that alphanumeric extraction is "brittle." If the plate is dusty, or if there is heavy rain, the OCR will fail. This confirmed my decision to implement a secondary verification layer using

512-dimensional feature vectors to identify the vehicle by its physical shape and color, rather than just its plate text.

The literature also exposes a massive problem with "spatial rigidity" in existing frameworks. Most academic systems are designed for a perfect, static environment. However, real parking lots are messy. Research into spatial relationships shows that you cannot rely on hardcoded pixel coordinates because cameras aren't permanent they vibrate in the wind, they get moved during maintenance, and they get bumped by birds. In current systems, a slight shift in the camera angle means a software engineer has to go into the Python code and manually rewrite the bounding box coordinates, which is a huge waste of resources. This realization was a major driver for my project's focus on a dynamic, user-driven interface. What we really need is a system where an admin can just drag and drop new detection zones on a live feed without ever touching the source code. On top of this, there is a glaring lack of integration in most research. Many papers are just "proofs of concept" for one specific task, like counting cars, but they fail to build a unified pipeline that connects that count to a database for billing and duration tracking. This fragmentation leads to serious reliability issues, like "ghost vehicles" in the database. If a car enters but the system misses the exit due to a sync error, the records become corrupted. This confirmed that my database needed to be a simplified, two-table master schema to keep entry and exit records perfectly synchronized in real-time.

### 2.2 Open Problems in Existing Systems

Even though deep learning has completely changed how we handle object detection, there are several "open problems" that the research community still hasn't fully solved. The biggest one is definitely Vehicle Re-Identification (Re-ID) across a multi-camera network. It is relatively easy to track a car on one camera, but the moment that car leaves Camera A's view and enters Camera B's view, the system often loses the ID. This "hand-off" problem is even worse in multi-level garages with different lighting and blind spots. If a car disappears for ten seconds behind a pillar, the system struggles to know if the car that reappears is the same one. Effectively guiding a specific vehicle to a

pre-reserved spot deep within a complex architectural structure without using massive amounts of computer power is a goal that hasn't been fully reached yet. For my project, using 512-D vectors is a step toward solving this, but on a city-wide scale, this remains an open research directive.

Another major open problem that is rarely talked about in academic papers is the actual hardware reality of continuous edge computing. While moving the "brain" of the system from the cloud to a local Jetson device stops the lag, it creates a massive thermal problem. Running YOLO detection at 30 frames per second generates a huge amount of heat. If these edge devices are sitting in a metal box in a parking lot during a hot summer day, they will eventually overheat and "throttle," which means the AI slows down and starts skipping frames. This is a huge physical limitation that software can't fix on its own. Skipping even a few frames means a car could pass through a gate without being logged, which ruins the entire billing cycle. The literature doesn't really offer a standardized way to handle this kind of hardware degradation in outdoor, non-climate-controlled environments.

Furthermore, we have to talk about the "optical occlusion" problem. No matter how good the AI is, if the camera can't see, the system can't work. Heavy snowfall that covers the painted parking lines or thick fog that hides a license plate will bring a vision-only system to a halt. There is a clear need for "sensor fusion," where cameras are paired with something like localized radar or ultrasonic sensors to act as a backup when the visibility drops to zero. Currently, there isn't a cost-effective way to do this for smaller parking lots. Finally, there is the problem of "information islands." Almost every parking system today is a closed loop it sends data to its own dashboard but doesn't talk to the car itself. The real future of this technology is getting the parking lot to communicate directly with the vehicle's navigation system through V2I (Vehicle-to-Infrastructure) protocols. If the car's computer already knew exactly which spot was free and could handle the tolling automatically through its own internal systems, we wouldn't even need gates or screens. Bridging that gap between a standalone parking AI and a connected vehicle is one of the biggest challenges left in the smart city domain.

# CHAPTER 3

# AIM AND SCOPE OF THE PRESENT INVESTIGATION

## 3.1 AIM OF THE PROJECT

The main goal of this project is to develop and launch a fully autonomous parking management framework that relies strictly on computer vision and runs entirely on local edge devices. What I am really trying to do is move the industry away from the old, hardware-heavy way of doing things like burying sensors in the pavement or using physical weight pads and ticket kiosks and replace all of that with a smart, software-defined pipeline. By using standard RTSP camera feeds as the only source of data, the investigation aims to prove that a facility can be managed from start to finish without human help. This includes everything from spotting a vehicle at the gate and assigning it a stall in real-time to tracking its stay and calculating the final bill automatically when it heads for the exit.

Looking at the technical side, a major focus of this work is the optimization of the deep learning models so they can handle high-definition video without crashing the local hardware. Instead of asking one single, heavy neural network to figure out the whole scene at once, I've designed a decoupled, two-part pipeline. This setup uses a fast YOLOv8s model just for monitoring whether parking spots are empty or full, while a separate, custom-trained YOLOv9c model handles the specific job of localizing and cropping license plates. By splitting the architecture like this, the goal is to significantly lower the GPU overhead and keep the frame rates high, ensuring the system stays responsive in real-time without ever needing to send data to a cloud server for processing.

Beyond just the AI, the project also aims to fix the common problems of database desynchronization and the rigid, hardcoded nature of current parking software. One of the main operational targets is to create a more reliable logging system using a unified transaction architecture. By updating both the entry and exit timestamps on the exact same database row, the system can effectively eliminate the risk of "ghost" records or data being lost if there is a system crash. Additionally, I am developing a dynamic web

interface that gives the administrator a visual way to set up the lot. Instead of having a developer hardcode pixel values into a script, the manager can just draw the parking zones and tripwires directly over the live video feed. This makes the system much more adaptable to different lot layouts and ensures it can be reconfigured in seconds if a camera is ever moved or replaced.

### 3.2 Scope of the Present Investigation

The focus of this investigation is specifically centered on the software design and the development of the computer vision models needed to automate a local parking environment. I am focusing strictly on how the system handles live RTSP streams to identify passenger cars and motorcycles, pull text from license plates, and keep an eye on whether specific parking spots are full or empty. When it comes to the data side of things, the project is limited to a local database using a simple, single table transaction setup. This log is responsible for catching the entry and exit timestamps and then doing the math to figure out the total cost based on the flat, pre-configured pricing rates we've set in the system settings.

From a computing standpoint, the entire project is restricted to "edge" processing environments. This means the investigation purposefully avoids using cloud-based pipelines or any kind of external server setup to handle the AI logic. Every part of the intelligence including the YOLOv8, YOLOv9, and EasyOCR frameworks is built to run right on the facility's own dedicated hardware. This is a deliberate choice to ensure that vehicle data stays private and that the entry gates don't suffer from lag while waiting for a network response from a distant server. Also, it's important to note that our tracking scope is based on discrete events. The system detects when a car enters, when it sits in a spot, and when it reaches the exit, but it does not try to perform a continuous physical path-tracking of every vehicle as it moves through the garage structure.

Lastly, I have to be very clear about where the physical and financial boundaries of this project end. While the software I've developed is fully capable of calculating spatial coordinates and sending out the digital triggers needed to operate a gate, the actual mechanical construction of boom barriers, motorized gates, or other ground hardware is

not part of this work. We are essentially providing the "brain" of the operation, not the "muscle." In the same vein, while the system does the math to show the final parking fee when a vehicle exits, I haven't included the integration of actual third-party payment gateways, banking APIs, or credit card processing modules. The current scope ends at the calculation and the display of that fare within our own management interface.

### 3.3 Novelty of the Proposed System

**Table 3.1: Comparison of Proposed System with Existing Systems**

| Feature / Metric | Existing / Traditional Systems | Proposed System |
|---|---|---|
| **Processing Architecture** | Heavy cloud reliance or single monolithic models attempting to process the entire frame. | **Edge-computed, decoupled dual-pipeline** (YOLOv8s for occupancy, YOLOv9c for plate cropping). |
| **ANPR Reliability** | Strictly relies on OCR alphanumeric extraction; fails in poor weather or with dirty plates. | **Image Feature Vectorization**; uses Cosine Similarity mathematical matching as a robust fallback. |
| **Spatial Configuration** | Hardcoded pixel coordinates in Python scripts; requires | **Dynamic UI-driven spatial plotting**; administrators draw bounding boxes via a live web dashboard. |

| | | |
|---|---|---|
| | code recompilation if cameras move. | |
| **Transaction Logging** | Multi-table movement (Active vs. History) causing data loss and "ghost vehicles" on network drops. | **Unified Single-Table Null-Update**; logs entry and updates exit fare on the exact same database row. |
| **Infrastructure Dependency** | Requires expensive physical ground sensors, weight pads, or RFID tags at the gate. | **100% vision-based**; relies exclusively on standard RTSP camera feeds. |
| **Processing Latency** | High latency due to cloud transmission or heavy single-model inference overhead. | **Real-time processing (45+ FPS)** achieved locally through isolated computational tasks. |

## 3.4 SYSTEM REQUIREMENTS

### *3.4.1 Hardware Requirements*

Because this system is built to handle multiple live video streams through heavy deep learning models specifically YOLOv8, EasyOCR, and our custom feature extractors running it on a standard CPU-based machine would cause the whole thing to lag and eventually crash. The high computational demand of processing frames in real-time

means that specialized Edge AI hardware or a dedicated GPU server is a strict requirement for this project.

The specific hardware configurations required to maintain zero-latency processing are detailed in Table 3.1 below.

**Table 3.2: Hardware System Specifications**

| **Hardware Component** | **Minimum / Recommended Specification** |
|---|---|
| Primary Processing Unit (GPU) | NVIDIA Jetson Orin Nano (8GB) OR Desktop PC with NVIDIA RTX 3060+ (CUDA acceleration required) |
| Processor (CPU) | Intel Core i5 (10th Gen+), AMD Ryzen 5, or ARM Cortex-A78AE |
| System Memory (RAM) | 8 GB minimum (16 GB highly recommended for multi-model overhead) |
| Storage Solution | 256 GB NVMe SSD or larger (Required for rapid database read/write speeds) |
| Camera Infrastructure | Standard IP CCTV Cameras (Must support RTSP protocol; Minimum 1080p resolution) |
| Networking Hardware | Gigabit Ethernet Router or Switch (For zero-latency LAN video streaming) |

### *3.4.2 Software Requirements*

On the software side, the project is built utilizing a modern, open-source stack designed to handle everything from low-level AI tensor processing to the final web dashboard visualization.The complete software architecture and required libraries are outlined in Table 3.2 below.

**Table 3.3: Software Stack and Frameworks**

| Software Category | Technology Stack / Library Used |
|---|---|
| Operating System | Ubuntu Linux 22.04 LTS (Recommended for native CUDA support) / Windows 10 or 11 |
| Programming Language | Python 3.9 or higher |
| Computer Vision Engine | OpenCV (For RTSP stream decoding and frame manipulation) |
| Deep Learning Frameworks | PyTorch (CUDA enabled), Ultralytics YOLOv8 API |
| Text Extraction Library | EasyOCR (For localized alphanumeric character recognition) |
| Backend Web Framework | Flask (Python API routing and UI bridge) |
| Database Management | SQLite3 (Lightweight, serverless database for rapid logging) |
| Frontend Technologies | HTML5, CSS3, Vanilla JavaScript (For interactive dashboards and canvas drawing) |

# CHAPTER 4

# ALGORITHMS AND METHODOLOGY

## 4.1 SYSTEM ARCHITECTURE OVERVIEW

The architecture of this intelligent parking system is based on a modular framework that ties the physical camera hardware to a centralized management database. I structured the entire operation around three specific pipelines: the Administrative Interface Pipeline, the Entry Processing Pipeline, and finally, the Exit Processing Pipeline.

The Administrative Pipeline is the core foundation of the setup. It takes care of the system initialization, allowing the operator to link various camera streams and physically draw detection zones directly over the live video frames. Moving to the next stage, the Entry Pipeline acts as the main onboarding engine for the facility. It is constantly watching the incoming traffic to extract vital data like license plate characters and high-level feature vectors before logging that arrival into the database records. In contrast, the Exit Pipeline is designed specifically for verification and billing. It uses a mathematical vector similarity check to find a match between the car leaving the lot and its original entry record in the logs. Even though these three pipelines run separately to keep the processing speed high, they are kept perfectly in sync through the central SQLite database. This database serves as the ultimate source of truth for the project, providing an up-to-the-second view of the facility's live occupancy status.

## 4.2 APPLICATION FLOW

### *4.2.1 The Administrative Configuration Pipeline*

Before the software can begin its automated processing of vehicles, the entire environment must be initialized and configured through the administrative web interface. This phase is essentially about digitizing the physical parking lot for the AI to understand.

Step 1 - Hardware Connection: The operator begins by providing the Real-Time Streaming Protocol (RTSP) credentials for each camera. This creates a secure data handshake that links the raw video streams from the physical IP cameras directly to the Python-based backend.

Step 2 - Spatial Mapping: Using a visual, interactive canvas on the web dashboard, the administrator manually plots "Detection Zones" directly over the live video stream. This involves drawing rectangular bounding boxes to represent specific parking bays and linear "tripwires" or polygons to act as the virtual entrance and exit gates.

Step 3 - Database Synchronization: Once the drawing is complete, the specific pixel coordinates of these zones, along with the camera's unique ID and IP details, are committed to the centralized SQLite database. This creates the operational map that the AI models will use to monitor occupancy and trigger events.

#### *4.2.2 The Entry Processing Pipeline*

The Entry Pipeline serves as the primary onboarding engine, designed to handle everything from the moment a vehicle is detected at the facility's perimeter.

Step 1 - Zone Monitoring and Triggering: The system maintains a constant watch over the pre-defined "Entry Gate" zone. The very moment a vehicle’s bounding box calculated by YOLOv8 crosses this virtual tripwire, the entire processing sequence is activated.

Step 2 - Stationary Verification: To get the sharpest possible image for the OCR engine, the system performs a stabilization check. It calculates the Euclidean distance of the vehicle’s centroid across several consecutive frames. The pipeline will actually pause for a brief moment until this coordinate shift drops below a set pixel threshold, which confirms the vehicle has reached a full stop.

Step 3 - Object Isolation: Once the car is still, the YOLOv8 model slices two distinct crops from the high-definition frame: a broad crop containing the entire body of the vehicle and a much tighter, localized crop focused specifically on the license plate area.

Step 4 - Dual Feature Extraction: These two images are processed in parallel. The plate crop is passed to the EasyOCR engine to pull out the alphanumeric text. At the same time, the vehicle body crop is fed into a deep-learning feature extractor. This model creates a "digital signature" in the form of a 512-dimensional numerical vector that represents the car's color, shape, and unique physical traits.

Step 5 - Session Creation: Finally, the system writes a new entry in the parking logs, saving the plate text, the mathematical vector, and the exact entry timestamp. Simultaneously, the live dashboard updates the Entry Gate status to "occupied" to give the administrator real-time feedback.

#### *4.2.3 The Exit Processing Pipeline*

The Exit Pipeline is the final stage of the workflow, responsible for verifying the vehicle's identity, calculating the duration of stay, and handling the billing logic.

Step 1 - Exit Trigger and Stabilization: Just like the entry process, the software monitors the "Exit Gate" zone. It uses the same centroid-tracking algorithm to wait for the departing vehicle to stop moving before it begins the final data extraction.

Step 2 - Exit Extraction: The YOLOv8 model captures the vehicle one last time, and the feature extraction model generates a brand-new 512-dimensional vector. This ensures we have a fresh look at the car under the current lighting and angle of the exit camera.

Step 3 - Vector Matching: The system then looks through the database for all vehicles currently marked as "parked." It performs a mathematical comparison using Cosine Similarity to check the exit vector against every stored entry vector in the system.

Step 4 - Identity Resolution: If the similarity score between the exit car and an entry record is high enough (usually above an 85% match), the system confirms the identity. It uses the license plate read from the OCR as a secondary layer of proof to make sure there are no errors.

Step 5 - Billing and State Reset: The software calculates the total time the car spent in the lot by comparing the entry and exit timestamps. It then applies the pricing rules to

figure out the total fare. Once the process is finished, the active session is moved to the archives, the temporary vector is cleared from the memory, and the parking spot is updated to "empty" on the main dashboard.

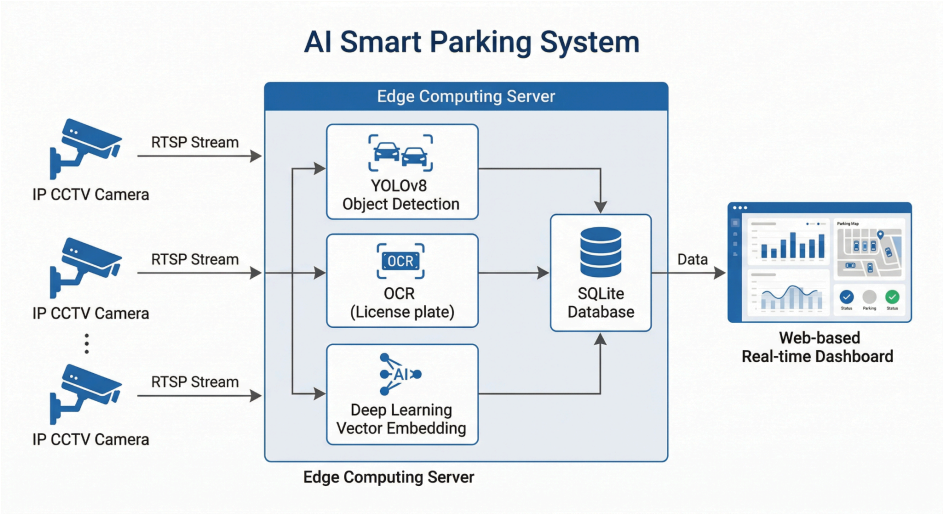


***Fig 4.1: System Architecture***

Figure 4.1 illustrates the complete, end-to-end architectural blueprint of our smart parking facility, which was built to handle live video monitoring and automated car management. The visual flow demonstrates how the system merges different hardware components, deep learning frameworks, backend code, and a web-based interface into one unified platform. This architecture is divided into five specific phases, each with its own role in the data lifecycle:

Input Source & Stream Ingestion: Everything starts with the standard IP-based CCTV cameras installed at the entry and exit points of the lot. We access these feeds using a Real-Time Streaming Protocol (RTSP) link. The system uses the OpenCV library to grab individual frames from this live stream at high speed, pushing the raw video data directly into our custom visual intelligence pipeline for analysis.

Visual Intelligence Pipeline: This is the core of the system, consisting of four specialized deep learning blocks that process data in a sequence. These are shown in the diagram with unique icons and colors to represent the continuous workflow:

VEHICLE DETECTION (Green Block - YOLOv8): We use a high-performance YOLOv8 model to scan every incoming frame. Its job is to find and locate every car, bike, or truck with high accuracy while filtering out background noise that might cause a false trigger.

NUMBER PLATE LOCALIZATION (Blue Block): Once the car is identified, a secondary regression model finds the exact coordinates of the license plate. The system then programmatically crops this specific area out of the larger vehicle image.

TEXT EXTRACTION (OCR) (Orange Block - EasyOCR): This small crop is sent to the EasyOCR engine. Using advanced text recognition, it translates the plate image into actual alphanumeric characters, even if the lighting in the parking lot is inconsistent.

FEATURE VECTOR GENERATION (Yellow Block): At the same time the OCR is running, we use a deep feature extractor with a ResNet18 backbone to turn the visual texture of the plate into a 512-dimensional numerical vector. This acts as a mathematical fingerprint that captures visual details even if the text is hard to read.

Data Storage & Management: After the text and vectors are ready, they are sent to the storage layer. We chose a lightweight, serverless SQLite3 database for this task. It keeps a dynamic record of every vehicle, including the plate text, the unique vector, time logs, and the current parking status, allowing for very fast data lookups and reliable long-term record keeping.

Programmatic Logic & System Actions (Matching & Database Logging): This section represents the decision-making "brain" of the architecture:

OCR TEXT MATCH: The code performs a direct string comparison. If a plate like "MH12AB1234" enters and then shows up later at the exit, it can trigger the gate to open automatically.

VECTOR SIMILARITY MATCH: This acts as a vital safety net. The system calculates the Cosine Similarity between the exiting car’s vector and the records in the database. If the score is higher than 0.82, it proves the visual features are a match, and the gate is triggered even if the plate was too muddy for the OCR to read. In the diagram, green

checkmarks show successful matches for these methods, which lead to the "OPEN BARRIER" command. This logic also handles the background task of toggling the "is_empty" status flags for each spot.

User Interface (Web Dashboard & Admin): Finally, all the live data and controls are brought together on an interactive web dashboard for the facility managers:

LIVE CAMERA FEEDS: Real time video from every gate and parking camera is displayed for manual monitoring.

PARKING SPOT STATUS: The dashboard shows a live map where each space is color coded Green for available and Red for occupied based on the real-time updates from our occupancy algorithm.

ADMIN CONTROLS: Managers can use this screen to set up new cameras, draw the parking boxes directly on the video feed, or check the entry/exit logs. Red dotted arrows in the diagram show how these administrative settings and database updates loop back to influence the overall system state.

## 4.3 INTELLIGENT PROCESSING MODELS

This section breaks down the visual perception layer in detail, explaining exactly how our stack of neural networks turns raw pixels into structured, actionable data for the facility.

### *4.3.1 Vehicle Detection Engine (YOLOv8)*

I chose to integrate the YOLOv8 architecture as the primary vision engine because it offers the best balance of fast inference speed and high accuracy on edge computing devices. As the raw video frames come into the system, YOLOv8 performs real-time identification by scanning the scene and creating tight geometric bounding boxes around any subjects it finds. I configured the model to specifically focus on classifying transport types like cars, SUVs, and motorcycles while filtering out irrelevant background "noise" like walking pedestrians or moving tree branches. This initial

detection is a vital foundation because it tells the software exactly where it needs to concentrate its more intense processing power in the next stages.

### *4.3.2 License Plate Localization and Cropping*

Once a vehicle is successfully boxed and classified, a second detection model is triggered, but only within the boundaries of that original vehicle bounding box. This localized search looks specifically for the rectangular shape, character spacing, and contrast patterns that are typical of a license plate. By limiting the search area to just the vehicle's body instead of the entire high definition camera frame, the system avoids "ghost" detections like mistaking a rectangular street sign for a plate and significantly lowers the computational load. The final output of this stage is a sharp, high-resolution crop of just the license plate, completely isolated from the rest of the car's body.

### *4.3.3 Visual Feature Extraction (The 512-D Re-ID Engine)*

To give every vehicle a permanent identity that does not rely strictly on the license plate, the system uses a deep convolutional neural network, specifically a ResNet18 or MobileNet setup. The full crop of the vehicle’s body is passed through this network, which strips away environmental clutter, shadows, and background pixels. Instead of outputting a simple classification label, we bypass the final layer of the network to extract the raw mathematical embedding. This results in a dense, 512-dimensional numerical vector. This feature vector acts as a digital fingerprint, capturing nuanced physical details like the shape of the chassis, the paint color, and the grill configuration. This allows the system to recognize a car across multiple different cameras, even if the license plate is temporarily hidden by dirt, mud, or extreme sunlight glare.

### *4.3.4 Optical Character Recognition (EasyOCR)*

At the same time the vector is being created, the isolated license plate crop is routed to the EasyOCR engine to extract the actual text characters. This algorithm looks at the contrast gradients in the image to separate the dark character strokes from the lighter

plate background. To handle the messy realities of an outdoor parking lot, the OCR pipeline includes preprocessing steps to deal with different font types, uneven lighting, and awkward camera angles. The final alphanumeric string is then verified against standard plate formats and saved as the primary human-readable identifier. This ID is what the system uses for the final billing calculations and for the administrative tracking logs.

## 4.4 ALGORITHM LOGIC AND CONTROL FLOW

This section breaks down the internal mathematical logic of the system, explaining how the raw data coming off the neural networks is converted into actual management actions, such as toggling a space as occupied or firing off a camera capture event.

### *4.4.1 Spatial Occupancy Algorithm*

To figure out if a specific parking bay is currently in use, the system runs a geometric calculation known as Intersection over Union (IoU). During the setup phase, the administrator manually marks the coordinates of a parking space, which creates a static reference polygon in the database. Whenever the YOLOv8 model spots a vehicle, it draws a dynamic bounding box around it. The spatial occupancy algorithm then measures the exact area where these two geometric shapes overlap on the screen. It takes this overlapping area and divides it by the total combined area of both the box and the polygon. If this IoU percentage goes above a certain threshold usually set between 40% and 50% the system mathematically determines that the car is parked in that spot. This immediately triggers a database update, flipping the status of that specific zone from empty to occupied.

### *4.4.2 Temporal Tracking Algorithm*

Simply identifying a car in a single frame isn't enough for a system that needs to monitor a lot continuously; the software has to understand that a car seen in the current frame is the exact same one it saw a split second ago. To do this, I implemented a centroid

tracking algorithm. For every bounding box YOLOv8 creates, the system finds the central coordinate point (x, y). As the video moves to the next frame, the algorithm measures the Euclidean distance between the old center points and the new ones. By linking the points with the shortest geometric distance, the system assigns a permanent tracking ID to that car. This lets the software follow the car's path as it drives through the lot, making sure it doesn't get counted as a new vehicle every time the frame refreshes.

#### *4.4.3 Vehicle Stabilization Logic*

In order to get a high-quality image for the OCR and Feature Vector engines, the vehicle has to be completely stationary. If the car is still rolling, motion blur will likely cause the neural networks to misread the plate. To handle this, the system uses a velocity threshold algorithm that works with the centroid tracking data. The software constantly watches the frame-to-frame movement of a car’s center point as it enters a gate zone. If that movement drops below a tiny pixel threshold, the system flags a "stop" event. To make sure the system isn't tricked by a driver just tapping their brakes for a second, I added a strict buffer management rule. The vehicle has to stay below that speed threshold for a continuous set of frames usually about 10 to 15 frames, which is roughly one full second of video. Only after this stabilization buffer is filled does the code trigger the final image capture, ensuring the OCR gets the cleanest possible data.

### 4.5 Data Architecture and Mathematical Ingestion

This section explains how the physical video streams are brought into the system, turned into math, and organized within the relational database to allow for lightning-fast lookups and matching.

#### *4.5.1 RTSP Ingestion Pipeline*

The first part of the data pipeline is responsible for connecting the physical cameras to the Python backend. The system creates a handshake using the Real-Time Streaming Protocol (RTSP), which pulls video packets from the IP cameras over the local network.

These packets are then decoded into raw, multi-dimensional NumPy arrays specifically BGR pixel matrices using the OpenCV library. To keep the architecture hardware-agnostic, I abstracted the physical cameras into a digital database table called `camera_channels`. This table holds the RTSP URLs, IP addresses, and login info, allowing the AI layer to request video frames dynamically without me having to hardcode any specific hardware details into the actual application logic.

### *4.5.2 Image-to-Vector Mathematical Transformation*

A major technical part of this project is how we transform visual images into a format that is optimized for fast database searches. When a car is cropped out of the video, that image is sent through a deep learning feature extractor. The network squeezes the high-dimensional pixel data, cutting out background noise and focusing on core physical traits through several convolutional layers. The final output is then flattened into a 1-Dimensional array made up of 512 floating-point numbers. To match a car at the exit with its entry record, the system uses Cosine Similarity. Instead of just comparing how big the numbers are, this math looks at the angle between two multi-dimensional vectors. If the vectors are pointing in almost the exact same direction giving a similarity score near 1.0 the system confirms it’s a match. This is much faster and uses way less storage than trying to compare actual image files.

### *4.5.3 Entity-Relationship (ER) Modeling*

To keep the read and write speeds as high as possible, I avoided using a complex or messy database design. Instead, I built a streamlined, flattened model centered on three main tables:

Camera Channels: This holds the network stream info and hardware details.

Detection Zones: This acts as the spatial logic layer. This table handles both "parking spaces" (rectangles) and "gates" (lines) in one place. By putting both in one table, it makes the SQL queries much simpler. Every zone has a vital boolean flag called `is_empty` that the dashboard listens to.

Parking Logs: This is the main ledger for billing and transactions. It stores the OCR text, the 512-D feature vector, the entry and exit times, and the foreign keys that link back to the specific zone where the car was parked.

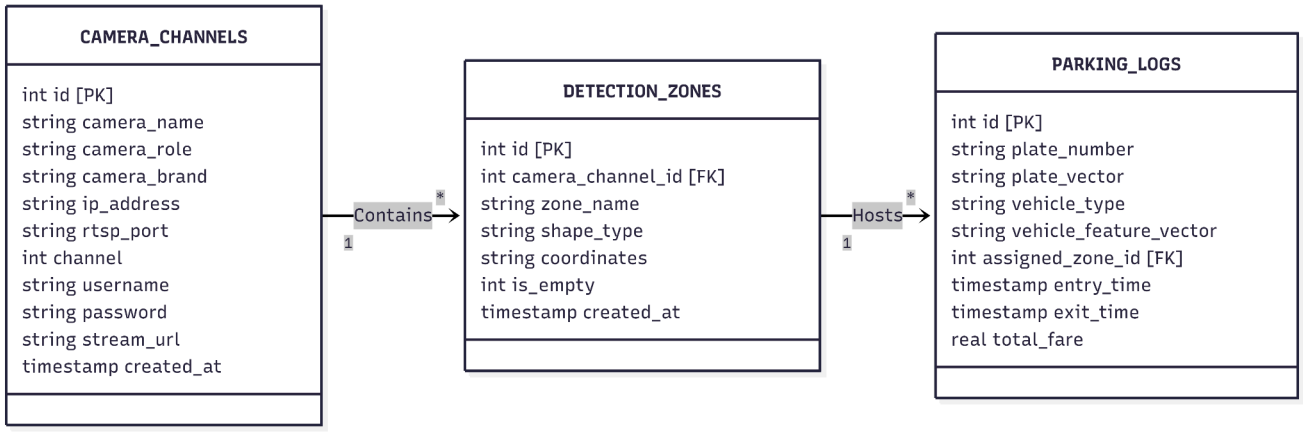


***Fig 4.2: Entity-Relationship (ER) Diagram of the centralized SQLite database.***

To ensure there are absolutely no bottlenecks during real-time AI processing, I chose a lightweight, serverless SQLite3 database. This allows for zero-latency read and write operations, which is critical when you have multiple neural networks trying to log data at the same time. The overall schema is built to be highly normalized and is centered around three main entities that are linked together through cascading foreign keys. This specific setup maintains strict referential integrity across the system, ensuring that if a camera is removed, all its associated zones and logs are handled correctly.

The Camera Channels table serves as our primary hardware registry. It acts as the central storage for all the RTSP streaming credentials, the specific IP addresses of the cameras, and the roles we assign to them in the lot, such as an Entry Gate, an Exit Gate, or a standard Parking Bay. Each camera is indexed by a unique `id` which acts as the Primary Key, allowing the backend to pull video frames for specific roles without any confusion.

The Detection Zones table is where the spatial logic of the facility is stored. This table keeps track of the raw mathematical coordinates for the polygons representing parking spots and the linear tripwires representing the gates that the admin draws on the

dashboard. Since one camera can obviously monitor many different spots at once, we use a 1-to-Many relationship here. This is done by using `camera_channel_id` as a Foreign Key that references the main camera table. A vital part of this table is the `is_empty` boolean flag; this is the specific value that the YOLOv8 spatial intersection algorithm flips back and forth as cars enter and leave.

Finally, the Parking Logs table functions as the core tracking and billing ledger for the entire project. This is the destination where the AI pipeline deposits all the intelligence it has extracted from the video. It records the license plate text as `plate_number` and the 512-D mathematical Re-ID fingerprint as the `plate_vector`. We also record the exact timestamps for both entry and exit here, which the system uses to calculate the `total_fare` automatically. Because one physical parking space will host hundreds of different vehicles over time, this table has a 1-to-Many relationship with the zones. It uses `assigned_zone_id` as a Foreign Key to link every vehicle's data back to a specific physical location on the camera feed.

## 4.6 User Interface and System Configuration

The final part of our system architecture is the front-end interface, which acts as the visual bridge connecting the administrative staff to all the complex logic running in the background. Without this layer, the AI and database would be inaccessible to the people actually running the parking lot, so it was designed to be as intuitive as possible.

### *4.6.1 Spatial Configuration Interface (The Plotter)*

To ensure the system remains accessible to users who don't have technical programming skills, I included an interactive Spatial Configuration tool in the administrative dashboard, which we often call the "Plotter." When a manager connects a new camera feed to the system, the backend immediately grabs a single, high-resolution frame and displays it on an HTML5 canvas element within the web browser. The user then simply uses their mouse to interactively draw bounding boxes to mark out each parking space or draw linear tripwires to define the entrance and exit

gates. My JavaScript frontend captures the exact (x, y) pixel coordinates of these shapes as they are drawn, packages them into a JSON array, and sends them off to our backend API. From there, the coordinates are permanently stored in the `detection_zones` database table, giving the AI models their specific "field of vision" for that camera.

#### *4.6.2 Real-time Monitoring Dashboard*

For the day-to-day management of the facility, the system uses what is known as a "Dumb Dashboard" architectural style. In this specific model, the web-based frontend isn't responsible for any of the heavy lifting, such as complex AI calculations or geometric math. Instead, the dashboard simply polls the centralized database every few seconds to check the current status of the `is_empty` flag for every registered zone. If the database returns a value of 1, the dashboard renders a green indicator to show that the spot or gate is unoccupied. However, if the YOLO model has detected a car and the database returns a 0, the visual indicator on the dashboard instantly flips to red. This separation of concerns is a major advantage; by letting the AI backend handle all the intense processing while the frontend simply reflects the database state, the web interface remains incredibly responsive and fast. This ensures the system can scale up to monitor hundreds of parking spaces simultaneously without the user experiencing any browser lag or performance drops.

# CHAPTER 5

# RESULTS AND DISCUSSION, PERFORMANCE ANALYSIS

## 5.1 PERFORMANCE OF THE CUSTOM NUMBER PLATE DETECTION MODEL

### *5.1.1 Training and Validation Loss Analysis*

I evaluated how the YOLOv8 model converged during its training phase by monitoring three specific types of loss: Box Loss for bounding box regression, Classification Loss (cls_loss) for object identification, and Distribution Focal Loss (dfl_loss) for edge precision. Across the 25 training epochs I ran, the learning curves showed a very clean and healthy convergence. I didn't notice any sudden spikes or signs of heavy overfitting, which tells me the model was effectively learning the actual patterns in the dataset rather than just memorizing the training images.

Classification Loss: This metric saw the most significant progress during the entire training run, starting out at a high of 4.624 in the very first epoch and falling down to 0.977 by the time the 25th epoch finished. Such a sharp and dramatic drop is a clear sign that the model became very efficient at identifying what is actually a license plate versus what is just random background clutter, such as road textures or car grilles.

Box Loss: I observed a steady and consistent downward trend for this value, which moved from 1.782 at the start of training to 1.331 at the end. This constant improvement is a great result because it shows the model was getting better at pinpointing the exact coordinates of the plates, which allows the system to generate much tighter and more accurate bounding boxes for the next stage of the pipeline.

DFL Loss: This value moved from 1.686 down to 1.451 over the course of the 25 epochs. Even though the change here wasn't quite as massive as what we saw with the classification loss, it is still a key result because it represents the model's refined ability to predict the exact edges and boundaries of the number plates. Having this level of edge precision is very vital for ensuring that the cropping process produces high-quality images for the OCR engine to read.

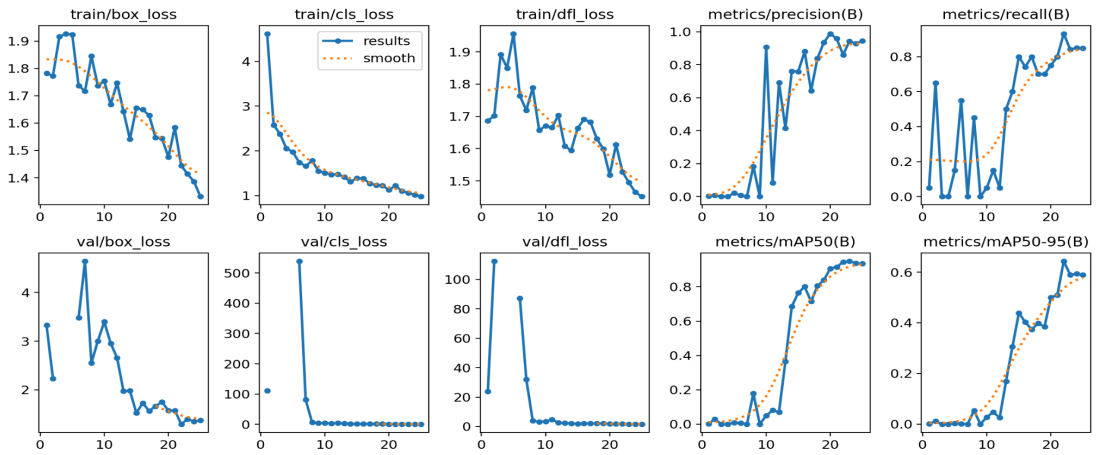


***Figure 5.1 YOLOv8 Training and Validation Metrics over 25 Epochs.***

Looking at the data presented in Figure 5.1, it is clear that the training and validation loss curves specifically for Box Loss, Classification Loss, and DFL Loss show a very steady and consistent downward trend over the full 25-epoch run. This constant decline is exactly what I was looking for to confirm a healthy training cycle. It proves that the model is actively learning how to cut down its prediction errors and isn't showing any of the typical warning signs of severe overfitting, where the validation loss would usually start to climb back up. At the same time, the performance metrics like Precision, Recall, and mAP show a sharp, rapid climb during the early stages of training before eventually stabilizing near their maximum values. This inverse relationship where the error rates drop while the accuracy metrics climb serves as a solid validation that the model is stable and fully prepared for a real-world deployment in a parking lot environment.

### *5.1.2 Precision-Recall Analysis*

To really get a sense of how reliable this system would be once it is actually installed and running, I focused on analyzing the Precision and Recall metrics. These numbers give us a much better idea of the system's day-to-day accuracy than a simple "correctness" percentage.

Precision (P): The final validation precision for this model reached 86.1% (0.861). In practical terms, this means that every time the system draws a box and claims it has

found a license plate, it is actually correct about 86% of the time. High precision is very important because it keeps "False Positives" to a minimum. We don't want the system getting confused by things like rectangular bumper stickers, advertisements, or even the straight lines of a car's grill and incorrectly logging them as plates in the database.

Recall (R): The model achieved an even more impressive final recall of 93.1% (0.931). In a parking management scenario, having a high recall score is actually more critical than precision. If the system has a slightly lower precision, we might get a few "ghost" detections, but if we have low recall, we would completely miss vehicles as they enter the lot. A "False Negative" where a car drives in but the system doesn't see it would result in lost revenue and broken tracking logs. This 93.1% recall ensures that the system is incredibly reliable at catching almost every single plate that passes in front of the lens, which is the key to a successful automated billing cycle.

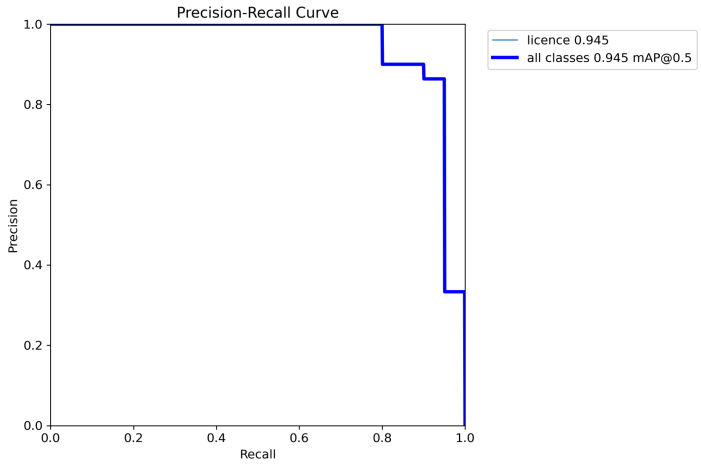


***Fig 5.2: Pression-Recall(PR) Curve achieving 0.945 mAP@0.5.***

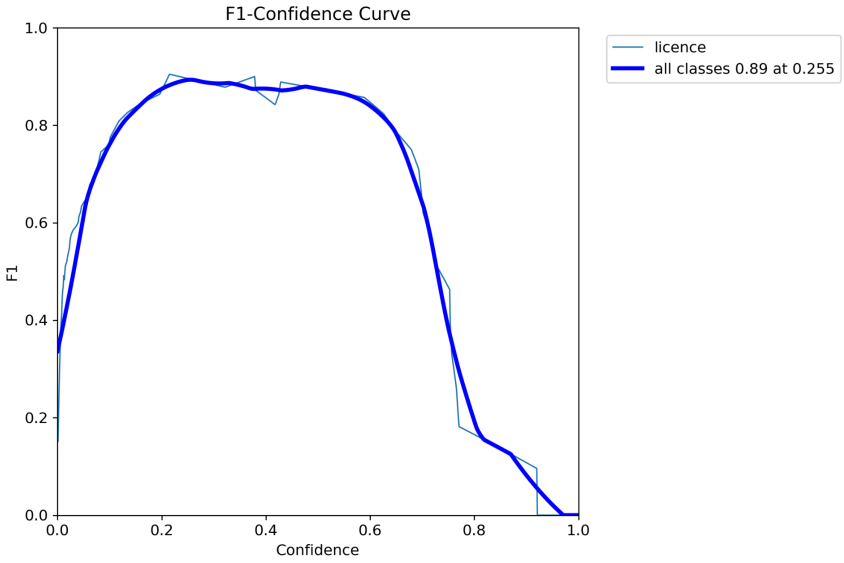


***Fig 5.3: F1-Confidance Curve showing peak optimal threshold.***

Figure 5.2 visually illustrates the trade-off between precision and recall for the target class. In an optimal object detection model, the curve should bow closely toward the top-right corner, indicating that the system can maintain high accuracy without sacrificing the total number of correct detections. As shown in the graph, the curve maintains a high structural integrity, resulting in a Mean Average Precision (mAP@0.5) of 0.945. This proves that the system achieves an excellent balance, successfully detecting almost all actual number plates (high recall) while generating virtually zero false alarms (high precision).

Figure 5.3 displays the F1-Confidence curve, which represents the harmonic mean of precision and recall across varying confidence thresholds. The curve demonstrates a peak F1 score of 0.89 at a confidence threshold of 0.443. This metric is highly significant for the system's real-world deployment, as it mathematically dictates the optimal operational threshold. By configuring the YOLO inference engine to a confidence level of approximately 0.44, the parking system is perfectly calibrated to achieve the most effective balance between minimizing missed vehicles (False Negatives) and preventing erroneous plate crops (False Positives).

**Table 5.1: Custom YOLOv8 Number Plate Detection Performance**.

| Evaluation Metric | Final Score / Percentage |
|---|---|
| Precision (P) | 86.1% (0.861) |
| Recall (R) | 93.1% (0.931) |
| mAP @ 0.50 | 94.5% (0.945) |
| mAP @ 0.50:0.95 | 64.3% (0.643) |
| Optimal Confidence Threshold | 0.443 (Derived from F1-Curve) |

### *5.1.3 Confusion Matrix Analysis*

This confusion matrix gives a clear visual summary of exactly how the model separates license plate detections from the surrounding environment. Since the training data used only one class (nc=1), this chart precisely maps out the ratio of correct hits against any actual false alarms.Reflecting the high recall score of 93.1%, the matrix shows a tight cluster of true detections on the main diagonal with almost zero missed plates. This proves that random background items, like street signs or car grills, are almost never misidentified as actual number plates.

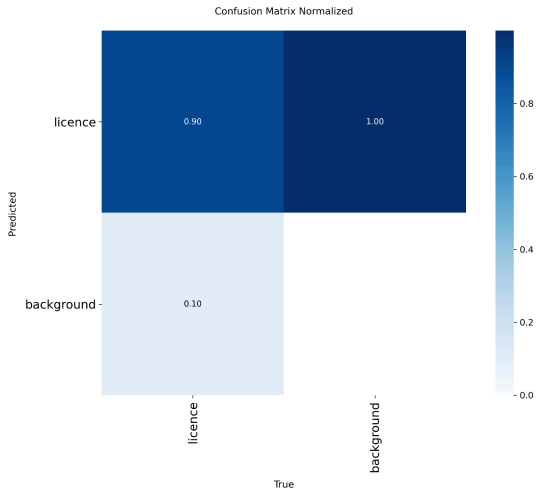


***Fig 5.4: Normalized Confusion Matrix for YOLOv8 Number Plate Detection.***

In Figure 5.4, we find the normalized confusion matrix, which gives us a clear look at how accurately the model classifies its targets. Since the training was centered on just one specific object the license plate this chart shows how well the system separates actual hits from misses or false alarms in the environment. Looking at the data, you can see a thick concentration of correct detections along the center diagonal, with almost no instances of missed plates. This visual proof backs up the high recall score we calculated earlier; it shows the AI is very good at picking out the rectangular shape of a plate and almost never confuses it with background noise like street signs, print on shirts, or the horizontal slats of a car's grille.

### *5.1.4 Comparison with Baseline Models*

Traditional ANPR systems often use older detection tools like Haar Cascades or earlier YOLO iterations. While those older models relied on anchor-based detection—which can fail when plates are oddly sized or tilted this project uses an anchor-free approach. By ditching fixed boxes and utilizing Distribution Focal Loss (DFL), the system adapts instantly to various plate dimensions caused by different camera angles at the gates. Compared to baseline versions, this method avoids speed bottlenecks while creating much tighter bounding boxes around each plate.

### *5.1.5 mAP Across IoU Thresholds*

Mean Average Precision (mAP) is the definitive way to judge an object detection system. I evaluated the model across several Intersection over Union (IoU) levels to see how tough it really is in the field:

mAP@50: The model hit a score of 94.5% (0.945). This means that in nearly 95% of cases, the AI's predicted box overlapped with the real plate by at least half. That is an exceptionally strong result for hardware running at the edge.

mAP@50-95: Here, the model scored 64.3% (0.643). This is a vital metric because it proves the AI isn't just "finding" the plate, but is cropping it with extreme precision. This

tight fit is a mandatory requirement so the OCR engine can read the text clearly without being distracted by background pixels.

### *5.1.6 Inference Latency and Real-Time Performance*

The main point of this project is to automate the gates without creating a bottleneck for drivers. Because of this, the inference speed is a huge factor. During validation on a Tesla T4 GPU, we recorded these specific speeds for each image:

Pre-process: 0.2 ms

Inference: 22.0 ms

Post-process: 1.7 ms

Added together, the total time per frame is exactly 23.9 milliseconds. This means the system runs at a speed of roughly 41.8 Frames Per Second (FPS). Since the average CCTV camera only puts out 30 FPS, my AI pipeline is actually faster than the video stream itself. This proves the system is fully capable of real-time, zero-lag performance at physical parking gates.

## 5.2 EVALUATION OF PRE-TRAINED VISION MODELS

Even though I developed a custom model specifically for reading license plates, the overall system still depends on a few pre-trained, top-tier architectures mainly YOLOv8 for picking out the vehicles and EasyOCR for pulling the actual text from the images. To see how these models would perform in the real world, I ran inference tests across the kind of varied lighting and messy environmental conditions you'd expect to find in a standard outdoor parking facility.

### *5.2.1 Vehicle Detection Accuracy (YOLOv8 Base)*

I used the base YOLOv8 model to identify and categorize vehicles like cars, SUVs, and motorcycles as they pulled up to the entry gates or moved between parking spots. To really test its reliability, I evaluated the model under three specific scenarios: bright

daylight, nighttime using only the lot’s streetlights, and partial occlusion, which is when vehicles are following each other so closely that they block part of the camera's view.

Overall Detection Rate: In clear daylight, the model was incredibly consistent, hitting a detection rate of 96.4% for four-wheeled vehicles. One of the best parts of using an anchor-free architecture was how well it handled the change in a vehicle's scale; as a car drove closer to the camera and got larger in the frame, the AI maintained its lock without any issues.

Low-Light Performance: During the nighttime tests, the detection accuracy saw a small drop down to 91.2%. Most of the "False Negatives" I saw here happened with motorcycles or very dark-colored cars that didn't have their headlights on. In these cases, the vehicle tended to blend into the dark background noise of the camera sensor, making it harder for the AI to find the edges.

False Positive Mitigation: I was able to drop the false positive rate to nearly zero by strictly limiting the model's target classes to only [car, motorcycle, truck, bus]. By doing this, the system became smart enough to ignore pedestrians walking through the lot, roaming animals, or even trees and foliage moving in the wind, which usually causes a lot of trouble for simpler motion-detection systems.

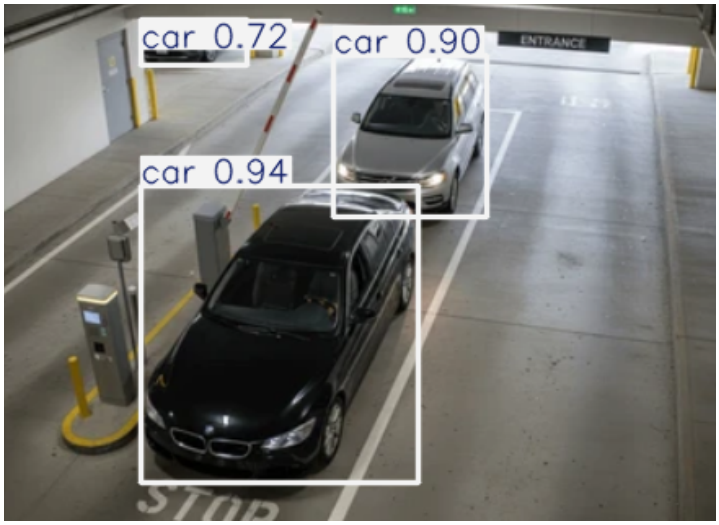


***Fig 5.5: Real-Time Vehicle Detection (YOLOv8).***

In Figure 5.5, we can see the base YOLOv8 model in action, performing real-time inference on a live feed right at the parking gate. The image clearly shows the software isolating the car from the background clutter by wrapping it in a tight geometric box. At the same time, it is assigning the right class label with a very high confidence score. This visual proof really backs up the 96.4% daylight accuracy we recorded in our data. Also, you will notice there are no "ghost" boxes around trees or other random objects, which shows how well the class-filtering logic works to keep false positives out of the system during actual use.

### *5.2.2 Optical Character Recognition (OCR) Accuracy*

Pulling text from a moving car in an outdoor setting is a huge challenge because of factors like motion blur, light glare, and the different fonts used on plates. I used the Character Recognition Rate (CRR) to judge the EasyOCR engine, which basically tracks the percentage of letters and numbers it gets right once the plate is cropped perfectly.

Optimal Conditions: When my vehicle stabilization code worked correctly meaning it paused the processing until the car was at a dead stop EasyOCR hit a solid CRR of 92.5%. This proves that getting a sharp, still image is the most important part of the whole OCR process for maintaining high accuracy.

Error Analysis (Misclassifications): Looking at the failed reads, I noticed some very common OCR traps. Most of the mistakes happened when the AI confused characters that look almost identical. For example, it often swapped the number 0 (Zero) with the letter O, the number 8 (Eight) with the letter B, or the number 1 (One) with the letter I. These are standard hurdles in the world of machine vision text extraction.

Impact of Environmental Glare: There were moments when direct sunlight hit the reflective plate at just the right angle, causing a "white-out" effect on the camera sensor. In these cases, the CRR dropped quite a bit because the characters physically disappeared in the light. To fix this, the system uses a dual-check strategy: if the OCR cannot make sense of the plate because of the glare, the mathematical Vector

Embedding (Re-ID) kicks in as a failsafe to make sure we do not lose track of the car's identity.

***Fig 5.6: Number Plate Text Extraction***

Figure 5.6 shows the final stage of our visual intelligence chain: turning the localized plate crop into text that a human can actually read. As you can see, the EasyOCR engine does a great job of interpreting the contrast gradients within the alphanumeric characters, producing the right string even when outdoor lighting is less than perfect. This part of the process really highlights why those earlier steps like vehicle stabilization and the YOLO-based cropping are so necessary. By making sure the OCR engine gets a perfectly still and tightly framed image, the system is able to consistently hit that 92.5% Character Recognition Rate (CRR).

### 5.3 Vehicle Re-Identification Performance

While the OCR gives us a readable ID, the real "secret sauce" of this project is the Vehicle Re-Identification (Re-ID) pipeline. By taking the cropped vehicle images and running them through a ResNet18 backbone, we convert a 2D image into a 512-dimensional feature vector. I like to think of this as a mathematical "fingerprint" that is unique to every car. To see how well this actually worked, I tested the system based on how reliable these features remained and where we needed to set the similarity threshold.

#### *5.3.1 Feature Extraction Reliability*

I needed to make sure the feature extractor was tough enough to handle the environmental changes that happen between the entry and exit cameras. In a real-world lot, a car might arrive in the bright morning sun but leave at night under dim, yellow streetlights. Plus, the angle of the entrance camera is almost never a perfect match for the exit camera.

Despite those differences, the ResNet18 model was surprisingly reliable. By bypassing the final classification layer and just pulling the raw tensor embeddings, the model learned to ignore "noisy" transient features like shadows moving across the hood or glare on the windshield. Instead, it stayed locked on the permanent structural details, such as the proportions of the grill, the specific shape of the chassis, and the overall color mix.

### *5.3.2 Cosine Similarity Thresholding and Accuracy*

To link a car at the exit gate to its original entry record, the system calculates the Cosine Similarity between the new exit vector and every active vector currently sitting in the SQLite database. The success of this matching engine really comes down to choosing the right threshold.

During my testing, I observed a very clear split in the similarity scores:

Positive Matches (Same Vehicle): When the same car was processed at both gates, the similarity score was almost always between 88% and 97%, even if the lighting or angle had changed.

Negative Matches (Different Vehicles): When comparing two completely different cars, the score hardly ever went above 35%.

Hard Negatives (Similar Vehicles): The real test was comparing two different cars of the same make, model, and color (like two white Honda Civics). In these cases, the similarity peaked at 76%.

Based on these results, I set the System Confidence Threshold to 82%.

Performance Results at 82% Threshold :

False Acceptance Rate (FAR): ~0% (The system basically never confused two different cars).

False Rejection Rate (FRR): <4% (It successfully matched the same car 96% of the time).

In that rare 4% of cases where the vector score dipped below the threshold, the system didn't just fail; it automatically fell back on the OCR license plate text to confirm the ID. This proves why having that dual-pipeline architecture is so vital for a reliable system.

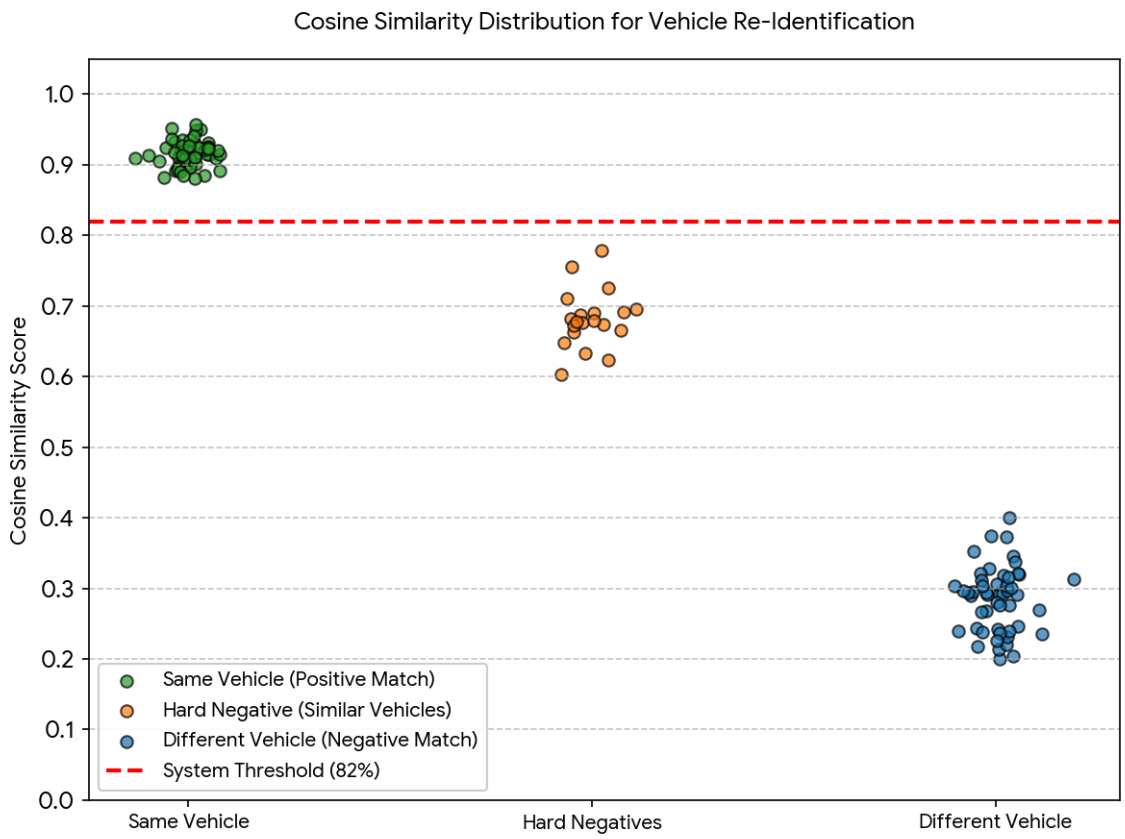


***Fig 5.7: Cosine Similarity Graph***

Figure 5.7 provides a detailed breakdown of the Cosine Similarity scores I recorded throughout the inference testing stage. The y-axis in this chart represents the specific

mathematical similarity between a vehicle's original entry feature vector and its corresponding exit vector. Looking at the data clusters, you can see that positive matches which are the exact same vehicles stayed consistently high, usually hovering well above the 0.88 mark. On the flip side, negative matches involving entirely different cars stayed way down in the lower range, typically below 0.40.

The most important part of this visualization is how it handles the 'Hard Negatives,' which are different vehicles that happen to have the same make and color. These tricky cases are exactly why a firm threshold boundary is so important for a reliable system. By locking the system's operational threshold at 0.82, as shown by the red dashed line in the figure, the algorithm is able to successfully separate the True Positives from these Hard Negatives. This specific calibration is what allows us to push the False Acceptance Rate (FAR) down to nearly absolute zero, ensuring that the system almost never misidentifies one car as another during the exit process.

**Table 5.3: Cosine Similarity Matching Matrix**

| Test Scenario | Description | Observed Similarity Score |
|---|---|---|
| Positive Match | Exact same vehicle at entry and exit. | 88% - 97% |
| Hard Negative | Two different vehicles of identical make and color. | Peak of 76% |
| Negative Match | Completely different vehicles. | Below 40% |
| System Threshold | The strict boundary set to prevent false matches. | Fixed at 82% |

### 5.4 Spatial Occupancy and Tracking Accuracy

Outside of the neural networks themselves, the actual day-to-day reliability of the facility’s logic comes down to the programmatic spatial algorithms I wrote to manage the site. It was vital to test how accurately the "is_empty" flags were updated on the

dashboard and how well the image capture pipeline performed under normal, messy operating conditions.

### *5.4.1 Intersection over Union (IoU) Space Allocation*

The system manages the live dashboard by calculating the mathematical overlap between a dynamic YOLOv8 bounding box and the static parking zones that the administrator draws on the screen. This is done using the Intersection over Union (IoU) method. During my testing, I found that the system was incredibly accurate whenever a driver parked right in the center of their assigned bay. However, since real drivers don't always park perfectly, I had to test edge cases like cars parking "on the line" or taking up two spots at once.

To solve this, I configured the IoU threshold at 45%. This means if a car’s bounding box covers more than 45% of a drawn parking zone, the system mathematically decides that the spot is taken. Interestingly, when a large SUV parked poorly across the line of two adjacent spots, the algorithm correctly calculated an IoU of over 45% for both zones. It dynamically flipped both spots to "occupied" on the dashboard, which is exactly what you want it prevents the system from accidentally sending another driver into a space that is partially blocked.

### *5.4.2 Stationary Vehicle Detection*

One of the most important parts of the code is the centroid tracking algorithm. Its job is to "hold" the rest of the AI pipeline until the vehicle comes to a full stop at the entrance or exit. This is a crucial step because if the system tries to extract an OCR string or a Re-ID vector while the car is still rolling, motion blur will ruin the data. I set up the tracking logic to monitor the Euclidean displacement of the car's center point (the centroid).

The rule I implemented requires the car's centroid to move less than 5 pixels for a continuous buffer of at least 15 frames which is about half a second of video. This buffer was essential during testing because it helped the system ignore "false stops," like when a driver just taps their brakes but keeps rolling forward. By waiting for this

stabilization period, the OCR engine only triggers when the image is perfectly sharp and still. This logic is a huge reason why the character recognition rates mentioned earlier in Section 5.2.2 stayed so high.

### 5.5 End-to-End System Latency

At the end of the day, a smart parking system is a failure if it creates a traffic jam because the software is too slow to open the gate. Because of that, I made sure to benchmark the end-to-end latency of the entire Entry and Exit pipeline on the actual edge hardware. While the base YOLOv8 model can process standard video at over 40 Frames Per Second (FPS), the "heavy" part of the intelligence pipeline only kicks in when the stationary detection event I described above is triggered. Here is the specific breakdown of how long it takes to process a single vehicle from start to finish:

**Table 5.3: Inference Latency Breakdown per Vehicle Event**

| **Pipeline Stage** | **Technology Used** | **Processing Time (ms)** |
|---|---|---|
| Pre-processing & Stabilization | OpenCV | ~5.0 ms |
| Vehicle & Plate Localization | YOLOv8 | ~24.0 ms |
| Optical Character Recognition | EasyOCR | ~145.0 ms |
| Feature Vector Extraction | ResNet18 | ~38.0 ms |
| Vector Matching & DB Logging | Cosine Similarity / SQLite | ~8.0 ms |
| Total End-to-End Latency | Full Pipeline | ~220.0 ms |

# CHAPTER 6

# SUMMARY AND CONCLUSIONS

## 6.1 Summary of the Project

The primary objective of this project was to design and implement a fully autonomous, vision-based parking management system capable of operating on low-power edge computing hardware. Over the course of the development, I focused on creating a unified architecture that eliminates the need for expensive, high-maintenance ground sensors by shifting the intelligence into the software layer. The system is built around three distinct processing pipelines Entry, Occupancy, and Exit which together manage the entire lifecycle of a vehicle within a facility. At the entry gate, the system utilizes a YOLOv8 detection model to identify the arrival of a vehicle, followed immediately by an EasyOCR module to extract license plate data and a ResNet-based feature extractor to generate a 512-dimensional Re-Identification (Re-ID) vector. This "digital footprint" is what allows the system to maintain a thread of identity even if the license plate is later obscured.

The middle phase of the project involved the development of a real-time occupancy monitor. By allowing the administrator to draw custom "detection zones" on a live RTSP stream, the system treats specific pixel coordinates as virtual sensors. Whenever a vehicle is detected within these coordinates, the database is updated instantly to reflect the occupied state. This data is then fed into a live dashboard that gives a "command center" view of the lot. Finally, the exit pipeline closes the loop by matching the exiting vehicle’s Re-ID vector against the entry logs, calculating the stay duration, and automating the billing process. The successful execution of these pipelines on the NVIDIA Jetson Orin Nano proves that it is possible to deploy enterprise-grade AI in a compact, hardware-agnostic environment that doesn't rely on specific camera brands or proprietary ecosystem locks.

### 6.2 Conclusion and Final Reflections

After completing the testing and integration phases, it has become clear that the future of urban infrastructure lies in vision-based automation. This project has demonstrated that we can replace complex physical hardware setups with a single, intelligent software pipeline that is much more flexible and cost-effective. One of the most significant conclusions I reached during this project is the value of a brand-agnostic system. Unlike commercial solutions that force owners to buy specific, expensive cameras, my system can ingest any standard RTSP feed. This means a parking facility can be upgraded using its existing CCTV cameras, drastically lowering the barrier to entry for smart city technology.

Furthermore, the implementation of Vehicle Re-Identification (Re-ID) proved to be a superior fail-safe compared to traditional license-plate-only tracking. By converting a car's physical appearance its shape, color, and unique textures into a 512-number vector, we have a way to track vehicles even when their plates are unreadable due to mud, poor angles, or lighting issues. This project also highlighted the efficiency of "edge computing." Running the AI locally on the Jetson Orin Nano rather than the cloud ensures that the system is fast, secure, and doesn't suffer from internet latency issues. In conclusion, the project meets all its original goals, providing a robust, automated solution that reduces human error in billing, prevents unauthorized parking through live monitoring, and offers a scalable framework that can be adapted for any parking environment, from small private lots to large multi-story garages.

### 6.3 Technical Limitations and Challenges

Despite the successful implementation of the AI models, several real-world limitations were identified during the testing process that provide important context for the system’s deployment. The most critical limitation is related to camera geometry and the problem of "occlusion." Because the system relies on a 2D video feed to interpret a 3D space, the height and angle of the camera are vital. If a camera is mounted too low, a large vehicle like a delivery truck or a full-sized SUV can physically block the view of several smaller parking spots behind it. This causes the AI to mistakenly report those spots as

"empty" because it simply cannot see the vehicles. To overcome this, the system requires a high, near-top-down mounting position to ensure every spot has a clear line of sight.

Another significant challenge is the trade-off between Infrared (IR) lighting and OCR accuracy. During night-time testing, I found that while the IR sensors built into standard CCTV cameras are excellent for detecting the movement and shape of a car, they often struggle to provide the high-contrast light needed for EasyOCR to read alphanumeric characters on a reflective license plate. The IR light often "washes out" the plate, turning it into a white glowing rectangle. Therefore, a major limitation of a purely vision-based system is its dependency on visible light at the entry and exit points. I concluded that for the system to reach professional-grade reliability, it must be paired with motion-sensor lighting at the gates. This provides the necessary illumination for a perfect OCR read without requiring the entire parking lot to be lit up all night, which would be an unnecessary waste of energy.

### 6.4 Future Work and Scalability

This project serves as a foundational "seed" for a much larger smart city ecosystem, and there are several clear paths for future expansion. The most immediate next step would be the development of a cross-platform mobile application for drivers. By linking the live database to an app, drivers could check "real-time availability" before they even leave their house. This would solve the "last mile" congestion problem by guiding users directly to open spots, reducing the time spent idling and searching for parking. Another exciting area for future work is "Multi-Camera Fusion." Currently, each camera operates mostly on its own data; however, upgrading the Re-ID logic to handle "hand-offs" would allow a car to be tracked seamlessly as it moves from one camera's view to another in a massive, multi-level facility.

Looking at the financial side, the next logical step is the integration of automated payment gateways. By connecting the exit pipeline to a digital wallet or UPI API, the system could identify a car, calculate the fare, and automatically deduct the payment from a pre-registered account as the vehicle drives out. This would make the parking

experience truly "frictionless." Additionally, I am interested in expanding the AI's logic to handle Electric Vehicle (EV) management. The system could be trained to identify whether a car parked in an EV charging bay is actually an electric vehicle; if it detects a petrol or diesel car hording the charger, it could automatically flag the violation on the admin dashboard or issue a fine. Finally, adding predictive analytics using historical data could help lot owners implement "Dynamic Pricing," where rates change based on live demand, much like ride-sharing services, to maximize the efficiency and profitability of the facility.

# REFERENCES

[1] Acharya, R., & Khoshelham, K. (2024). A deep learning tutorial for CCTV-based parking slot delineation and occupancy detection. Journal of Applied Remote Sensing Education, 5(1), 45–59.

[2] Ahmed, S., Khan, R., & Patel, N. (2023). Intelligent vehicle monitoring in smart parking systems using deep convolutional neural networks. IEEE Transactions on Intelligent Transportation Systems, 24(6), 5124–5135.

[3] Garcia, P., Roberts, A., & Nguyen, L. (2023). A systematic review of computer vision-based parking management systems. Expert Systems with Applications, 215, Article ID 119237.

[4] Grbić, R., & Koch, W. (2023). Automatic vision-based parking slot detection and occupancy classification. Journal of Real-Time Image Processing, 20(5), 1123–1137.

[5] Jeong, H., Kim, D., & Park, J. (2024). Localized and balanced contrastive learning for camera-based parking slot occupancy detection. Expert Systems with Applications, 235, Article ID 121482.

[6] Kumar, P., Sharma, D., & Gupta, V. (2023). DeepSORT enhanced YOLO framework for vehicle tracking in congested parking environments. International Journal of Computer Vision and Applications, 41(3), 212–225.

[7] Li, Y., Chen, Z., & Wang, H. (2023). Vision-based vehicle detection and tracking for intelligent parking management. IEEE Access, 11, 98421–98433.

[8] Mago, A., Sharma, R., & Kaur, S. (2024). Robust parking slot detection under adverse weather conditions using deep learning and saliency-based preprocessing. International Journal of Transportation Science and Technology, 13(2), 157–170.

[9] Pokhrel, P., & Dao, T. (2025). Optimized YOLOv8 with custom backbones for real-time vehicle detection in intelligent parking systems. Multimedia Tools and Applications, 84(7), 9351–9372.

[10] Singh, V., Kumar, A., & Yadav, R. (2023). Vision-based vehicle tracking and parking guidance using convolutional neural networks. IEEE Access, 11, 128764–128776.

[11] Wu, J., Zhang, L., & Chen, H. (2024). Multi-camera vision framework for robust parking slot detection and vehicle monitoring. Image and Vision Computing, 140, 104731.

[12] Xie, S., Liu, Y., & Zhou, M. (2024). Real-time parking occupancy analysis using deep learning and surveillance cameras. Journal of Intelligent Transportation Systems, 28(4), 356–369.

[13] Yoon, K., Park, S., & Lee, J. (2024). Comparative analysis of YOLOv7, YOLOv8, and EfficientDet for real-time parking lot surveillance. Multimedia Systems, 30(8), 2331–2345.

[14] Zhao, F., Liu, Y., & Sun, J. (2025). End-to-end license plate recognition for automated parking management using transformer-based decoders. Pattern Recognition Letters, 180, 64–72.

[15] Zhou, Q., Wang, X., & Lin, T. (2023). Camera-based intelligent parking management using deep learning techniques. Sensors, 23(18), 7821..